

UNIVERSIDAD DE COSTA RICA
SEDE PACÍFICO

ALGORITMOS Y ESTRUCTURAS DE DATOS

INFORMÁTICA
EMPRESARIAL

MATERIAL DIDÁCTICO



```
public class Nodo <T>{  
    private T dato;  
    private Nodo<T> siguiente;  
  
    public Nodo (T dato){  
        siguiente=null;  
        this.dato=dato;  
    }  
} // fin de la clase
```

ESTE TEXTO ESTÁ DIRIGIDO A QUIENES SE ESFUERZAN
POR APRENDER DÍA A DÍA, SIN IMPORTAR LAS
CONDICIONES ADVERSAS QUE IMPEREN

Tema Grafos

Marcos Venegas y Karol Sánchez

Un grafo está compuesto de vértices (también llamados nodos) y de relaciones entre vértices. Los grafos son muy utilizados en las matemáticas y en las ciencias de la computación. Aunque prácticamente cualquier problema puede representarse mediante grafos, pues representa relaciones binarias entre elemento.

En el siguiente material se explican los grafos, grafos dirigidos y grafos no dirigidos. Se abordan los conceptos de camino, grado, grados de entrada y de salida.

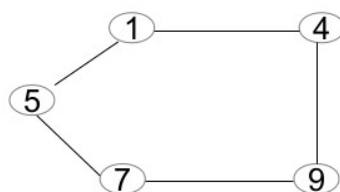
En lo que respecta a los grafos dirigidos se estudian las representaciones a través de la matriz de adyacencia y la lista de adyacencia. Se abordan el problema de los caminos más cortos con un solo origen y el problema de los caminos más cortos entre todos los pares. Para ello se exponen las soluciones propuestas por (Dijkstra, 1959) Dijkstra, que soluciona el primer problema y se explican los trabajos de (Floyd, 1962) Floyd y (Warshall, 1962) Warshall para resolver el segundo. Todos citados por (Aho et al., 1988) Aho.

En relación a los grafos no dirigidos también se estudian las representaciones a través de la matriz de adyacencia y la lista de adyacencia. Se realiza una introducción a árboles abarcadores de costo mínimo y se analizan las técnicas propuestas por (Prim, 1957) Prim, y (Kruskal, 1956) Kruskal para construirlos.

Grafos

Un grafo G agrupa entes físicos o conceptuales y las relaciones entre ellos. Está conformado por un conjunto de vértices o nodos V , que representa a los entes, y un conjunto de arcos A , que representan las relaciones entre vértices.

Se representa con el par $G = (V, A)$.



Grafo no dirigido

Muestra un grafo formado por:

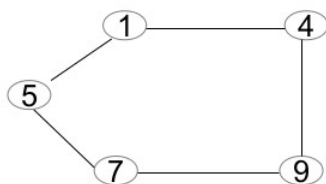
$V = \{1, 4, 5, 7, 9\}$

$A = \{(1,4), (4,1), (5,1), (1,5), (7,9), (9,7), (7,5), (5,7), (4,9), (9,4)\}$

Un arco o arista representa una relación entre dos nodos. Esta relación, al estar formada por dos nodos, se representa por (u,v) siendo u,v el par de nodos.

Un grafo es no dirigido, si los arcos están formados por pares de nodos no ordenados, no apuntados. Se representa con un segmento uniendo los nodos, $u-v$.

Ejemplo:

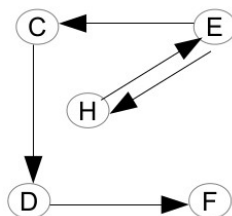


Grafo no dirigido

Un grafo es dirigido, también denominado digrafo, si los pares de nodos que forman los arcos son ordenados.

Se representa con una flecha que indica la dirección de la relación, $u \rightarrow v$.

Ejemplo:



Tipos de datos abstractos

Muestra un grafo formado por:

$V = \{C, D, E, F, H\}$

$A = \{(C,D), (D,F), (E,H), (H,E), (E,C)\}$

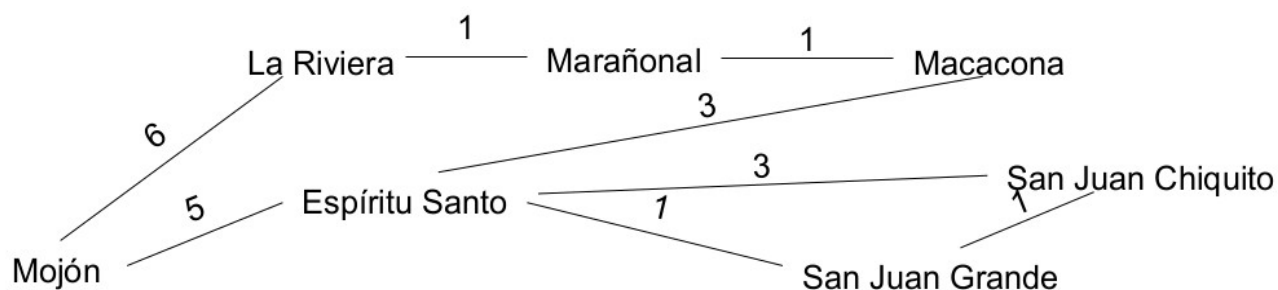
Dado el arco (u, v) de un grafo, se dice que los vértices u y v son adyacentes.

Grafo valorado

En los modelos realizados con grafos, a veces, una relación entre dos nodos tiene asociada una magnitud, denominada factor de peso, en cuyo caso se dice que es un grafo valorado.

Ejemplo:

Los pueblos que forman una comunidad junto con la relación entre un par de pueblos que están unidos por un camino: esta relación tiene asociado el factor de peso, que es la distancia en kilómetros.



Grado

El grado es una cualidad que se refiere a los nodos de un grafo.

En un grafo no dirigido, el grado de un nodo v , $\text{grado}(v)$, es el número de arcos que contienen a v .

Ejemplo:

$\text{Grado}(\text{Mojón}) = 2$.

Grado de entrada y salida

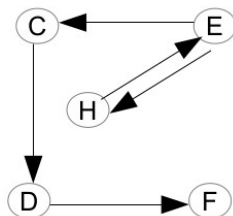
En un grafo dirigido se distingue entre grado de entrada y grado de salida:

- Grado de entrada de un nodo v , $\text{gradent}(v)$: es el número de arcos que llegan a v .
- Grado de salida de v , $\text{gradsal}(v)$: es el número de arcos que salen de v .

Tipos de datos abstractos

Ejemplo:

- Gradient (D) = 1
- Gradsal (E) = 2



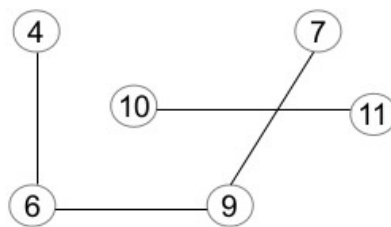
Camino

Un camino P de longitud n desde el vértice v_0 a v_n en un grafo G, es la secuencia de $n+1$ vértices $v_0, v_1, v_2, \dots, v_n$.

La longitud del camino es el número de arcos que lo forma.

Ejemplo:

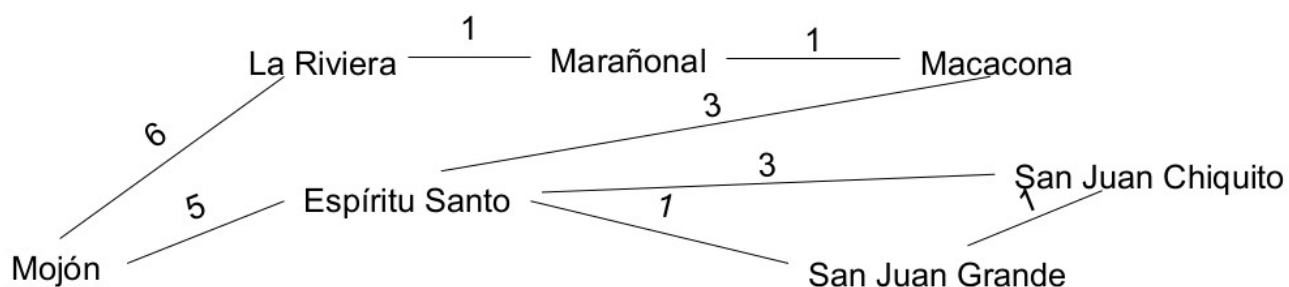
- $P_1 = (4, 6, 9, 7)$ es un camino de longitud 3.
- $P_2 = (10, 11)$ es un camino de longitud 1.



En un grafo valorado, la longitud del camino con pesos es la suma de los pesos de los arcos en el camino.

Ejemplo:

El camino (Mojón, Espíritu Santo, Macacona) tiene la longitud $5 + 3 = 8$.



Un camino es simple si todos los nodos que forman el camino son distintos.

En un grafo dirigido, un ciclo es un camino simple cerrado. Por tanto, un ciclo empieza y termina en el mismo nodo, $v_0=v_1$, y además, debe tener más de un arco.

Un grafo dirigido sin ciclos se denomina GDA = Grafo Dirigido Acíclico.

Grafos dirigidos

Un grafo dirigido G consiste en un conjunto de vértices V y un conjunto de arcos A . Los vértices de un grafo dirigido pueden usarse para representar objetos, y los arcos, relaciones entre los objetos. Por ejemplo, los vértices pueden representar ciudades, y los arcos, vuelos aéreos de una ciudad a otra. Otro ejemplo puede ser utilizar un grafo dirigido para representar el flujo de control en un programa de computadora, los vértices representan los bloques básicos, y los arcos, posibles transferencias del flujo de control.

Representaciones de grafos dirigidos

Para representar un grafo dirigido se pueden emplear varias estructuras de datos; la selección apropiada depende de las operaciones que se aplicarán a los vértices y a los arcos del grafo.

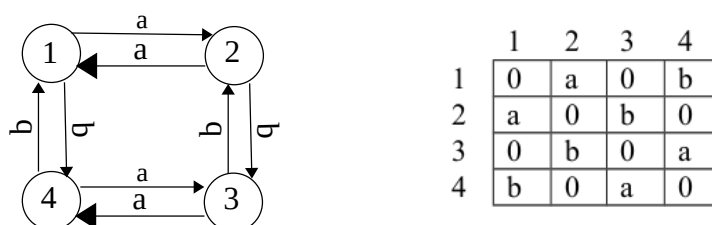
a. Matriz de adyacencia.

La matriz de adyacencia para G es una matriz A de dimensión $n \times n$, de elementos booleanos, donde $A[i,j]$ es verdadero si, y sólo si, existe un arco que vaya del vértice i al j . Con frecuencia se utilizarán matrices de adyacencias con 1 para verdadero y 0 para falso.

La representación con matriz de adyacencia es útil en los algoritmos para grafos, en los cuales suele ser necesario saber si un arco dado está presente.

Algo muy relacionado en este tipo de representación es la matriz de adyacencia etiquetada de un grafo dirigido, donde $A[i,j]$ es la etiqueta del arco que va del vértice i al vértice j . Si no existe un arco de i a j , debe emplearse como entrada para $A[i,j]$ un valor que no pueda ser una etiqueta válida.

Ejemplo de grafo dirigido con su representación de matriz de adyacencia etiquetada.



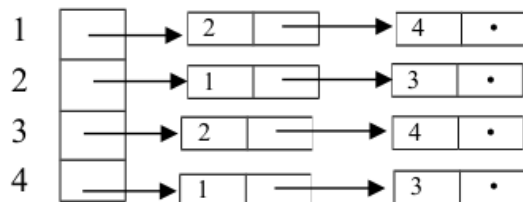
Una desventaja que tiene este tipo de representación es que tiene un tiempo de duración mayor. Solo examinar o leer la matriz puede llevar un tiempo $O(n^2)$.

b. Lista de adyacencia

Otro método de representación es la lista de adyacencia. La lista de adyacencia para un vértice i es una lista, en algún orden, de todos los vértices adyacentes a i .

La representación de un grafo dirigido con lista de adyacencia requiere un tiempo de $O(n)$.

Ejemplo:

**Problema de los caminos más cortos con un solo origen.**

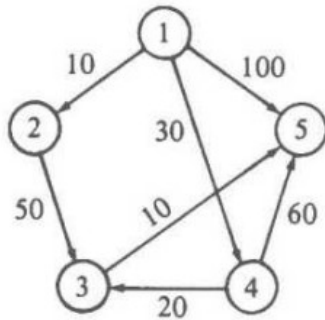
Supóngase un grafo dirigido $G=(V,A)$ en el cual cada arco tiene una etiqueta no negativa, y donde un vértice se especifica como origen. El problema es determinar el costo del camino más corto del origen a todos los demás vértices de V , donde la longitud de un camino es la suma de los costos de los arcos del camino. Esto se conoce con el nombre de problema de los caminos más cortos con un solo origen.

Sea G un mapa de vuelos en el cual cada vértice representa una ciudad, y cada arco $v \rightarrow w$, una ruta aérea de la ciudad v a la ciudad w . La etiqueta del arco es el tiempo que se requiere para volar de v a w . La solución del problema de los caminos más cortos con un solo origen para este grafo dirigido determinaría el tiempo de viaje mínimo para ir de cierta ciudad a todas las demás del mapa.

Para resolver este problema se manejará una técnica conocida como algoritmo de Dijkstra, que opera a partir de un conjunto S de vértices cuya distancia más corta desde el origen ya es conocida. En principio, S contiene sólo el vértice de origen. En cada paso, se agrega algún vértice restante v a S , cuya distancia desde el origen es la más corta posible. Suponiendo que todos los arcos tienen costo no negativo, siempre es posible encontrar un camino más corto entre el origen y v que pasa sólo a través de los vértices de S . En cada paso del algoritmo, se utiliza un arreglo D para registrar la longitud del camino especial más corto a cada vértice. Una vez que S incluye todos los vértices, todos los caminos son “especiales”, así que D contendrá la distancia más corta del origen a cada vértice.

El algoritmo supone que existe un grafo dirigido $G=(V,A)$ en el que $V=\{1,2,3 \dots, n\}$ y el vértice 1 es el origen. C es un arreglo bidimensional de costos donde $C[i,j]$ es el costo de ir del vértice i al vértice j por el arco $i \rightarrow j$. Si no existe el arco $i \rightarrow j$ se supone que $C[i,j]$ es ∞ , un valor mucho mayor que cualquier costo real. En cada paso, $D[i]$ contiene la longitud del camino especial más corto actual para el vértice i .

Grafo



Considere la siguiente matriz C:

	1	2	3	4	5
1	∞	10	∞	30	100
2	∞	∞	50	∞	∞
3	∞	∞	∞	∞	10
4	∞	∞	20	∞	60
5	∞	∞	∞	∞	∞

Inicialmente S contiene el origen.

En la primera iteración el vector D contendrá como valores iniciales la distancia del origen (vértice 1) a cada vértice si el arco existe. Si no tendrá el valor ∞ .

Iteración	S	w	D[2]	D[3]	D[4]	D[5]
Inicial	{1}	—	10	∞	30	100

Para seleccionar el vértice w a agregar en S se escoge el vértice con el mínimo valor en el vector D. El $\min(10, \infty, 30, 100)$ es 10 y pertenece al vértice 2. Por lo que se elige en la siguiente iteración como vértice w a 2, y se agrega al conjunto S. El nuevo valor en cada posición del vector D estará dado por la siguiente fórmula: $D[v] = \min(D[v], D[w] + C[w, v])$. Se consideran solamente aquellos vértices que no están en S, pues los que están en S ya están identificados como vértices con distancia mínima. Entonces, se omite la comparación para D[2].

w=2

Fórmula $D[v] = \min(D[v], D[w] + C[w, v])$

Para v=3 $D[3] = \min(D[3], D[2] + C[2, 3])$, (recuerde que arriba puede observar la matriz C[fila, columna])

$$D[3] = \min(\infty, 10 + 50)$$

$$D[3] = 60$$

Para v=4 $D[4] = \min(D[4], D[2] + C[2, 4])$,

$$D[4] = \min(30, 10 + \infty)$$

$$D[4] = 30$$

Para v=5 $D[5] = \min(D[5], D[2] + C[2, 5])$,

$$D[5] = \min(100, 10 + \infty)$$

$$D[5] = 100$$

Tipos de datos abstractos

Iteración	S	w	D[2]	D[3]	D[4]	D[5]
Inicial	{1}	–	10	∞	30	100
1	{1,2}	2	10	60	30	100

Para elegir el vértice w se debe encontrar el menor de elemento de la tabla, excluyendo los vertices que se encuentran en S. Por el momento solo se encuentran en S el origen y el vértice 2, el $\min(60, 30, 100)$ es 30. Entonces, el vértice w será 4 y se agrega al conjunto S. Se consideran solamente aquellos vértices que no están en S, pues los que están en S ya están identificados como vértices con distancia mínima. Entonces, se omite la comparación para D[2] y para D[4].

w=4

Fórmula $D[v] = \min(D[v], D[w] + C[w, v])$

Para v=3 $D[3] = \min(D[3], D[4] + C[4, 3])$, (recuerde que arriba puede observar la matriz C[fila, columna])

$D[3] = \min(60, 30 + 20)$

$D[3] = 50$

Para v=5 $D[5] = \min(D[5], D[4] + C[4, 5])$,

$D[5] = \min(100, 30 + 60)$

$D[5] = 90$

Iteración	S	w	D[2]	D[3]	D[4]	D[5]
Inicial	{1}	–	10	∞	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90

En esta iteración w es 3.

Fórmula $D[v] = \min(D[v], D[w] + C[w, v])$

Para v=5 $D[5] = \min(D[5], D[3] + C[3, 5])$,

$D[5] = \min(90, 50 + 10)$

$D[5] = 60$

Iteración	S	w	D[2]	D[3]	D[4]	D[5]
Inicial	{1}	–	10	∞	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60

Tipos de datos abstractos

Finalmente, w es 5.

Iteración	S	w	D[2]	D[3]	D[4]	D[5]
Inicial	{1}	—	10	∞	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60

Problema de los caminos más cortos entre todos los pares.

Suponga que se tiene un grafo dirigido etiquetado que da el tiempo de vuelo para ciertas rutas entre ciudades, y se desea construir una tabla que brinde el menor tiempo requerido para volar entre dos ciudades cualesquiera. El problema es encontrar el camino de longitud más corta entre v y w para cada par ordenado de vértices (v,w) .

Podría resolverse este problema por medio del algoritmo de Dijkstra, tomando por turno cada vértice como vértice origen, pero una forma más directa de solución es mediante el algoritmo de Floyd.

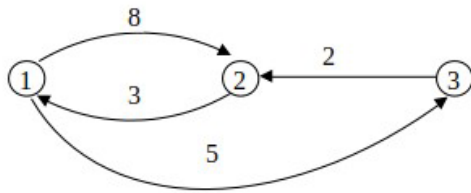
El algoritmo de Floyd usa una matriz A de $n \times n$, en la que se calculan las longitudes de los caminos más cortos. Inicialmente se hace $A[i,j] = C[i,j]$ para toda i diferente a j . Si no existe un arco que vaya de i a j , se supone que $C[i,j] = \infty$. Cada elemento de la diagonal se hace 0.

Después, se hacen n iteraciones en la matriz A . Al final de la k -ésima iteración, $A[i,j]$ tendrá por valor la longitud más pequeña de cualquier camino que vaya desde el vértice i hasta el vértice j y que no pase por un vértice con número mayor que k . Esto es, i y j , los vértices extremos del camino, pueden ser cualquier vértice, pero todo vértice intermedio debe ser menor o igual que k . En la k -ésima iteración se aplica la siguiente fórmula para calcular A .

En cada iteración se aplica la siguiente fórmula:

$$A_k[i, j] = \min \begin{cases} A_{k-1}[i, j] \\ A_{k-1}[i, k] + A_{k-1}[k, j] \end{cases}$$

El subíndice k denota el valor de la matriz A



C=

	1	2	3
1	2	8	5
2	3	∞	∞
3	∞	2	∞

 $A_0[i,j]$

	1	2	3
1	0	8	5
2	3	0	∞
3	∞	2	0

A_0 es la matriz inicial (la base para realizar el algoritmo de Floyd), es igual a la matriz de costos C , solo que la diagonal se hace 0. En todas las matrices, la diagonal siempre es 0.

A_1 (Para calcular A_1 , se mantiene la fila y la columna 1 de la matriz anterior)

	1	2	3
1	0	8	5
2	3	0	8
3	∞	2	0

 $A_1[2,3] = \min$ $A_0[2,3] = \infty$ $A_0[2,1] + A_0[1,3] = 3 + 5 = 8$ $A_1[3,2] = \min$ $A_0[3,2] = 2$ $A_0[3,1] + A_0[1,2] = \infty + 8 = \infty$ A_2

	1	2	3
1	0	8	5
2	3	0	8
3	5	2	0

 $A_2[1,3] = \min$ $A_1[1,3] = 5$ $A_1[1,2] + A_1[2,3] = 8 + 8 = 16$ $A_2[3,1] = \min$ $A_1[3,1] = \infty$ $A_1[3,2] + A_1[2,1] = 2 + 3 = 5$ A_3

	1	2	3
1	0	7	5
2	3	0	8
3	5	2	0

 $A_3[1,2] = \min$ $A_2[1,2] = 8$ $A_2[1,3] + A_2[3,2] = 5 + 2 = 7$ $A_3[2,1] = \min$ $A_2[2,1] = 3$ $A_2[2,3] + A_2[3,1] = 8 + 5 = 13$

Cerradura Transitiva

Supóngase que la matriz de costo C es sólo la matriz de adyacencia para el grafo dirigido dado. Esto es, $C[i,j] = 1$ si hay un arco de i a j , y 0 si no lo hay. También se conoce como cerradura transitiva de la matriz de adyacencia.

	1	2	3
1	0	1	1
2	1	0	1
3	1	1	0

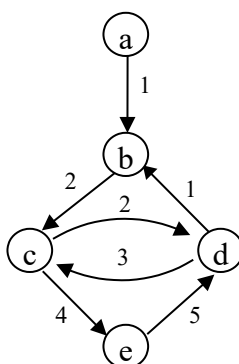
Se conoce también como
Algoritmo de Warshall

Localización del centro de un grafo dirigido

Supóngase que se desea determinar el vértice más central de un grafo dirigido. Este problema se puede resolver fácilmente con el algoritmo de Floyd. Primero se hace más preciso el término “vértice más central”. Sea v un vértice de un grafo dirigido $G=(V,A)$. La *excentricidad* de v es:

$$\max_{w \text{ en } V} \{\text{longitud mínima de un camino de } w \text{ a } v\}$$

El centro de G es un vértice de mínima excentricidad. Así, el centro de un grafo dirigido es un vértice más cercano al vértice más distante.



b	c	d	e	a
$a \rightarrow b = 1$	$a \rightarrow c = 3$	$a \rightarrow d = 5$	$a \rightarrow e = 7$	$b \rightarrow a = \infty$
$c \rightarrow b = 3$	$b \rightarrow c = 2$	$b \rightarrow d = 4$	$b \rightarrow e = 6$	$c \rightarrow a = \infty$
$d \rightarrow b = 1$	$d \rightarrow c = 3$	$c \rightarrow d = 2$	$c \rightarrow e = 4$	$d \rightarrow a = \infty$
$e \rightarrow b = 6$	$e \rightarrow c = 8$	$e \rightarrow d = 5$	$d \rightarrow e = 7$	$e \rightarrow a = \infty$

Se escogen los valores más altos de cada vértice.

Las excentricidades de los vértices son:

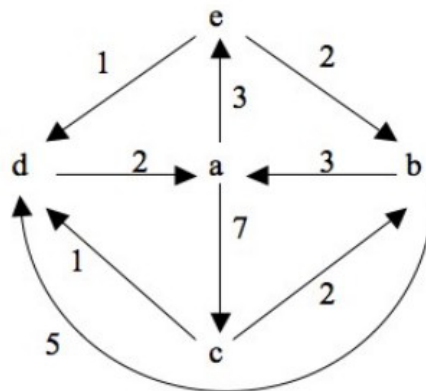
vértice	excentricidad
a	∞
b	6
c	8
d	5
e	7

Por tanto, el centro es el vértice **d**.

Se escoge el vértice con menor valor.

Práctica:

Dado el siguiente grafo:



1. Obtenga, por medio del algoritmo de Dijkstra, el camino más corto del vértice “a” hacia todos los demás.
2. Obtenga, por medio del algoritmo de Floyd, el camino más corto entre todos los vértices.
3. Encuentre el centro del grafo.

Grafos no dirigidos

Un grafo no dirigido $G=(V,A)$ consta de un conjunto finito de vértices V y de un conjunto de aristas A . Se diferencia de un grafo dirigido en que cada arista en A es un par no ordenado de vértices. Si (v,w) es una arista no dirigida, entonces $(v,w)=(w,v)$.

Los grafos se emplean en distintas disciplinas para modelar relaciones simétricas entre objetos. Los objetos se representan por los vértices del grafo, y dos objetos están conectados por una arista si están relacionados entre sí.

Buena parte de la terminología para grafos dirigidos es aplicable también a los no dirigidos. Por ejemplo, los vértices v y w son adyacentes si (v,w) ó (w,v) es una arista.

Un *camino* es una secuencia de vértices $v_1, v_2, \dots, v_n$ tal que (v_i, v_{i+1}) es una arista para $1 \leq i < n$. Un camino es *simple* si todos sus vértices son distintos, excepción de v_1 y v_n , que pueden ser el mismo. La longitud del camino es $n-1$, el número de aristas a lo largo del camino. Se dice que el camino $v_1, v_2, \dots, v_n$ conecta v_1 y v_n . Un grafo es conexo si todos sus pares de vértices están conectados.

Sea $G=(V,A)$ un grafo con conjunto de vértices V y conjunto de aristas A . Un *subgrafo* de G es un grafo $G'=(V',A')$ donde:

1. V' es un subconjunto de V .
2. A' consta de las aristas (v,w) en A tales que v y w están en V' .

Un ciclo (simple) de un grafo es un camino (simple) de longitud mayor o igual a tres, que conecta un vértice consigo mismo. No se consideran ciclos los caminos de la forma v (camino de longitud 0), v, v (camino de longitud 1), o v,w,v (camino de longitud 2).

Un grafo es cíclico si contiene por lo menos un ciclo. Un grafo conexo acíclico algunas veces se conoce como **árbol libre**.

Un árbol libre puede convertirse en ordinario si se elige cualquier vértice deseado como raíz y se orienta cada arista desde ella.

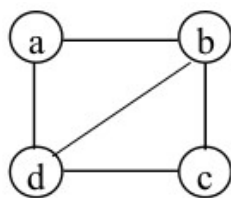
Los árboles libres tienen dos propiedades importantes:

1. Todo árbol libre con $n \geq 1$ vértices contiene exactamente $n-1$ aristas.
2. Si se agrega cualquier arista a un árbol libre, resulta un ciclo.

Métodos de representación

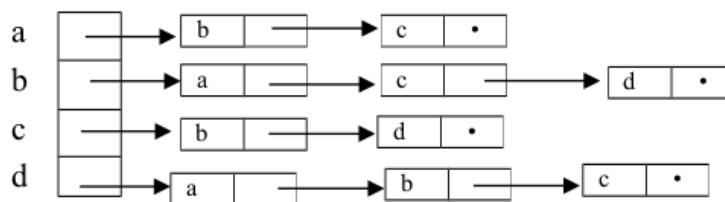
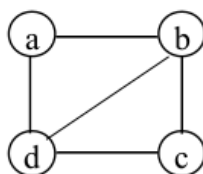
Los métodos de representación de grafos dirigidos se pueden emplear también para representar los no dirigidos. Una arista no dirigida entre v y w se representa simplemente con dos aristas dirigidas, una de v a w , y otra de w a v .

1. Matriz de adyacencia



	a	b	c	d
a	0	1	0	1
b	1	0	1	1
c	0	1	0	1
d	1	1	1	0

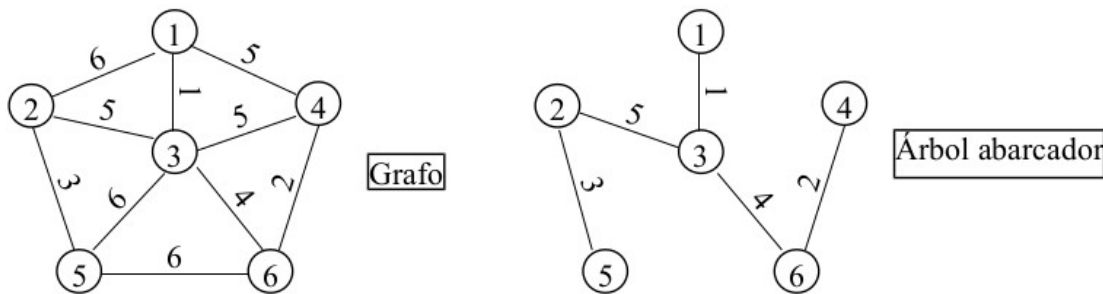
2. Lista de adyacencia



Árboles abarcadores de costo mínimo (AAM)

Supóngase que $G = (V, A)$ es un grafo conexo en donde cada arista (u,v) de A tiene un costo asociado $c(u,v)$. Un árbol abarcador para G es un árbol libre que conecta todos los vértices de V ; su costo es la suma de los costos de las aristas del árbol.

Una aplicación típica de los árboles abarcadores de costo mínimo tiene lugar en el diseño de redes de comunicación. Los vértices del grafo representan ciudades, y las aristas, las posibles líneas de comunicación entre ellas. El costo asociado a una arista representa el costo de seleccionar esa línea para la red. Un árbol abarcador de costo mínimo representa una red que comunica todas las ciudades a un costo minimal.



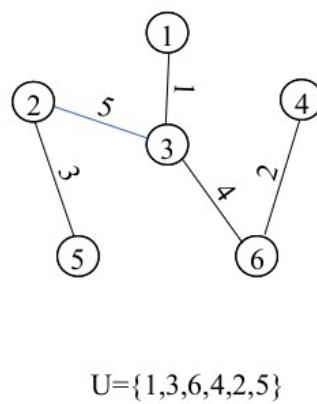
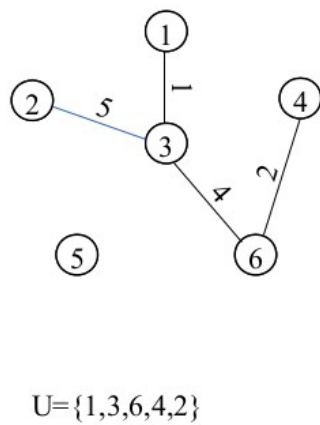
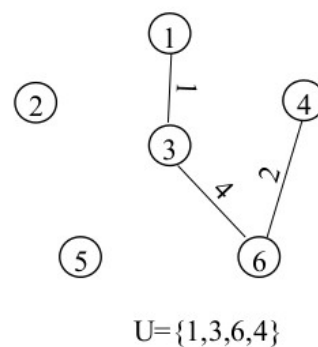
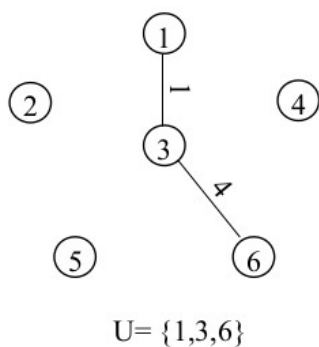
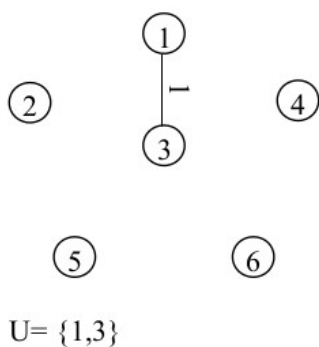
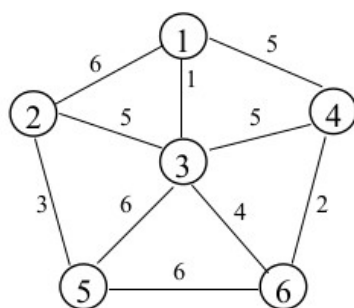
Hay distintas formas de construir un árbol abarcador de costo mínimo, en este curso se estudiarán:

- Algoritmo de Prim
- Algoritmo de Kruskal

Algoritmo de Prim

Existen dos técnicas populares que explotan la propiedad AAM (Árbol Abarcador de costo Mínimo) para construir un árbol abarcador de costo mínimo a partir de un grafo ponderado $G=(V,A)$; una de ellas se conoce como algoritmo de Prim.

Suponga que $V=\{1,2,\dots,n\}$. El algoritmo de Prim comienza cuando se asigna a un conjunto U un valor inicial $\{1\}$, en el cual “crece” un árbol abarcador, arista por arista. En cada paso localiza la arista más corta (u,v) que conecta U y $V-U$, y después agrega v , el vértice en $V-U$, a U . Este paso se repite hasta que $U=V$.



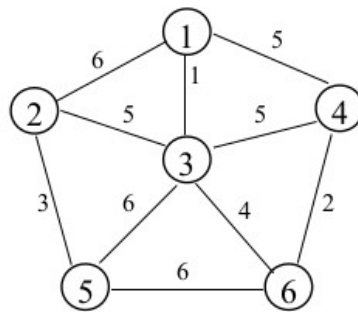
Costo mínimo
= 15

Algoritmo de Kruskal

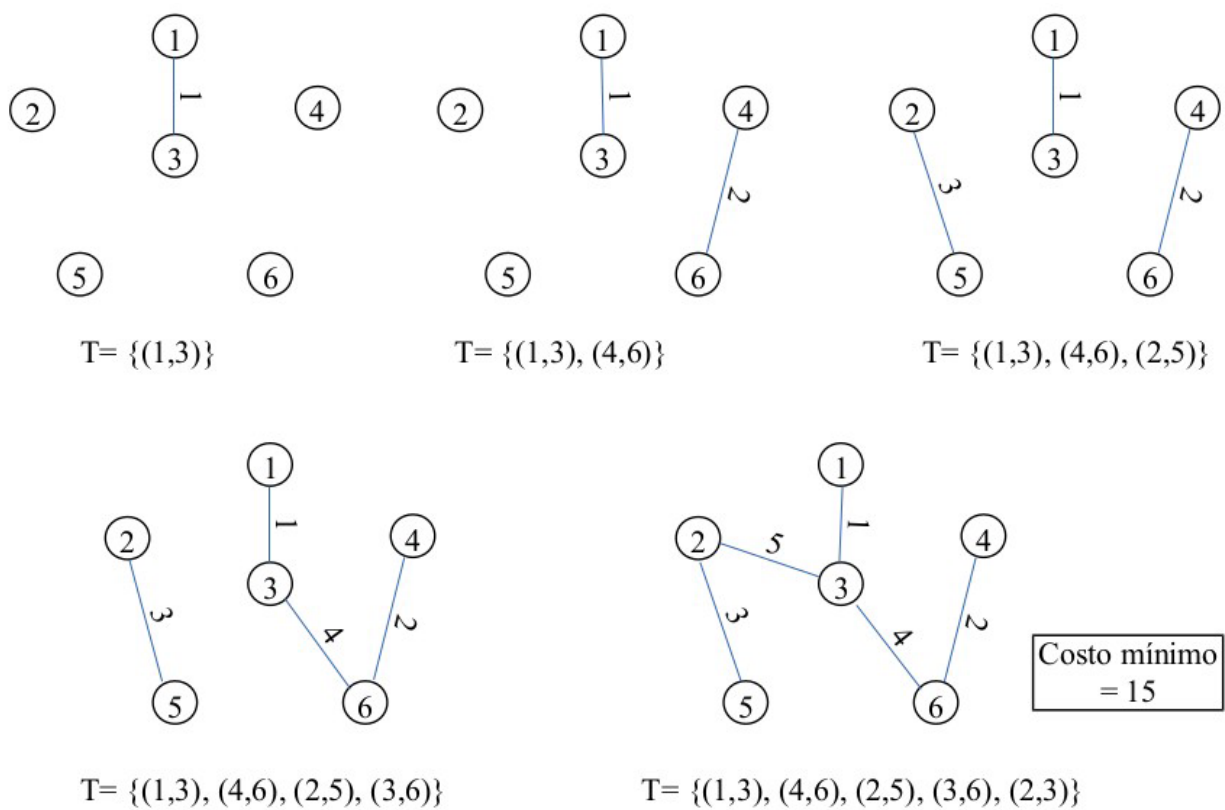
Sea $G=(V,A)$, con $V=\{1,2,...,n\}$ y un función de costo c definida en las aristas de A . Otra forma de construir un árbol de extensión de costo mínimo para G es empezar con un grafo $T=(V, \emptyset)$ constituido sólo por los vértices de G y sin aristas. Por tanto, cada vértice es un componente conexo por sí mismo. Conforme el algoritmo avanza, habrá siempre una colección de componentes conexos, y para cada componente se seleccionarán las aristas que formen un árbol de extensión.

Para construir componentes cada vez mayores, se examinan las aristas a partir de A , en orden creciente de acuerdo con el costo. Si la arista conecta dos vértices que se encuentran en dos componentes conexos distintos, entonces se agrega la arista T . Se descartará la arista si conecta dos vértices contenidos en el mismo componente, ya que puede provocar un ciclo si se la añadiera al árbol de extensión para ese componente conexo. Cuando todos los vértices están en un solo componente, T es un árbol de extensión de costo mínimo para G .

Ejemplo. Secuencia de aristas añadidas por el algoritmo de Kruskal.



Tipos de datos abstractos



Bibliografía

Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1988). Estructuras de datos y algoritmos (Vol. 1). Addison-Wesley Iberoamericana.

Birtwistle, G. M., Dahl, O. J., Myhrhaug, B., & Nygaard, K. (1973) SIMULA BEGIN. Filadelfia, Pensilvania. Auerbach Press

Cook, W. R. (2009, October). On understanding data abstraction, revisited. In Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications (pp. 557-572).

Guttag, J. V. (1975). The specification and application to programming of abstract data types. University of Toronto.

Liskov, B., & Zilles, S. (1974). Programming with abstract data types. ACM Sigplan Notices, 9(4), 50-59.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische mathematik, 1(1), 269-271.

Floyd, R. W. (1962). Algorithm 97: shortest path. Communications of the ACM, 5(6), 345.

Warshall, S. (1962). A theorem on boolean matrices. Journal of the ACM (JACM), 9(1), 11-12.

Prim, R. C. (1957). Shortest connection networks and some generalizations. The Bell System Technical Journal, 36(6), 1389-1401.

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. Proceedings of the American Mathematical society, 7(1), 48-50.